

Wayne State University
College of Engineering — Department of Electrical and Computer Engineering
Analysis of Ultrasound Distance Sensing System
Numerical Methods (ECE3040-Project #1)

Group Members:

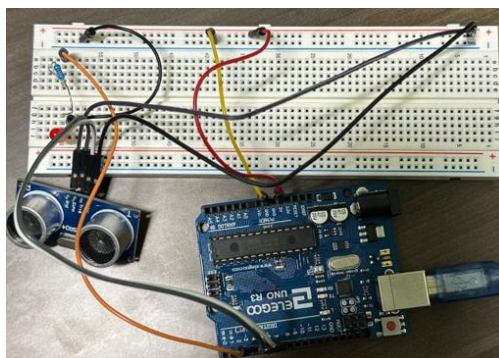
- **Mina George:** Data collection, Curve fitting
- **Abid Ahmad:** Error analysis, Documentation
- **Mustakim Choudhury:** Numerical modeling, Arduino setup

*This report was done with equal contribution from each member

Video: [YouTube Link](#)

Instructor: Abhilash Pandya

Date: 10/14/25



1. Introduction

This project is focused on the design and numerical analysis of a distance-sensing system. It is integrated with Arduino UNO and HC-SR04 Ultrasound sensor, and is all powered by the MATLAB App designer Interface. The goal of this project is to analyze the accuracy of the sensor and model, and correct the errors using regression analysis. Accurate distance measurement is essential in autonomous vehicle systems for reliable navigation and obstacle avoidance. This project evaluates the performance of the HC-SR04 ultrasonic distance sensor connected to an Arduino microcontroller, with data acquisition and analysis performed via a MATLAB-based Graphical User Interface (GUI). The GUI is developed using MATLAB's App Designer and Object-Oriented Programming (OOP) concepts to provide real-time visualization, data storage, and error analysis. The primary focus of the project is to identify and quantify errors in the sensor's distance measurements under different environmental conditions and surface materials. By collecting distance data at known reference points, the project explores error characteristics and implements linear, quadratic, and cubic polynomial models to correct systematic inaccuracies. These correction models are applied to both static measurements and dynamic tracking of moving objects

Additional features include continuous plotting of sensor data, LED indicators (both virtual and physical) to signal valid data capture, and numerical methods for velocity estimation using Newton's forward and central difference formulas. The project emphasizes well-structured, documented code and a user-friendly GUI that supports hardware and file-based data input. This report outlines the experimental setup,

methodology, error analysis, regression modeling, correction of sensor data, and velocity computation for moving targets. It also discusses the impact of environmental factors and material properties on sensor accuracy, offering insights into practical considerations for ultrasonic sensor deployment in autonomous systems.

2. System Design & Implementation

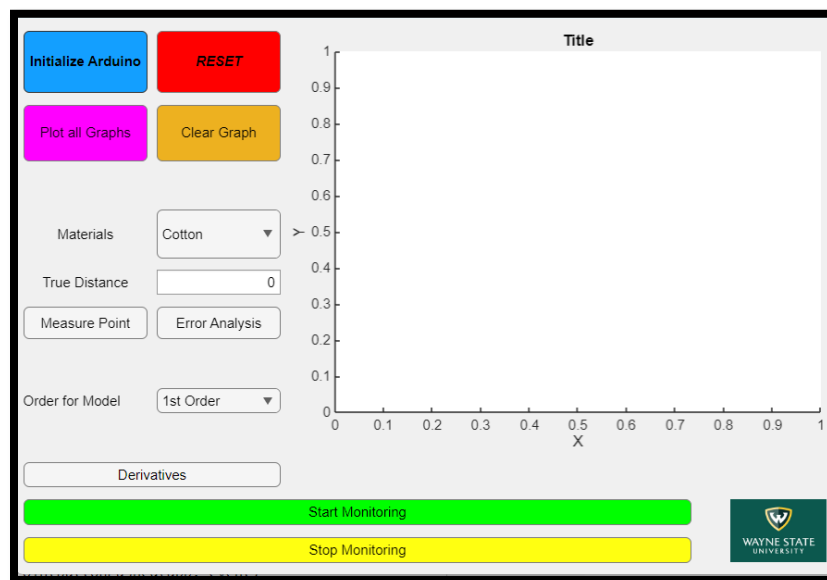
2.1 Hardware Setup (Component Used)

- ★ Arduino Uno Microcontroller
- ★ HC-SR04 ultrasonic sensor
- ★ LED indicator (PIN D3)
- ★ Breadboard and wires

Connections: Trig – D7 | Echo – D6 | LED – D3 | Vcc – 5V | GND – GND

2.2 Software and GUI implementation

The MATLAB APP designer interface has been used to create a user-friendly interface for all operations, like Arduino initialization, data collection, and graphing etc. The MATLAB programming environment has been used to build and verify the code for the ultrasound sensor.



2.3 Key GUI components

- ★ **Initialize Arduino**- establish the connection, link the 'DistanceSensor' Object, and prepare the sensor
- ★ **Start/Stop** - Starts and stops the live distance monitoring and plotting
- ★ **Measure point** – It uses this data as the measured distance value and compares it to the predetermined true distance, which is used to compare with the captured distance for calibration and error analysis
- ★ **Error Analysis**- Uses all the data captured to compute and graph the true, relative, and approximate errors and plots them

- ★ **Order for model-** It allows users to select their preferred model of regression. Models range from 1st to 3rd order
- ★ **Derivatives-** It computes the velocity and applies the forward/central/backward difference
- ★ **Material Section-** Allows the user to select up to 3 different materials for analysis

Code excerpt- Arduino Initialization

```
% Button pushed function: InitializeArduinoButton
function InitializeArduinoButtonPushed(app, event)
    %ABID
    %Our pins for the Arduino Board, we can update if needed
    port = 'COM3';
    board = 'Uno';
    trig = 'D7'; %pin 7
    echo = 'D6'; %pin 6
    led = 'D3'; %pin 3
    app.sensor = DistanceSensor(port, board, trig, echo, led);
    disp('Sensor initialized successfully.');
```

Code excerpt- Start Monitoring the Data

```
% Button pushed function: StartMonitoringButton
function StartMonitoringButtonPushed(app, event)
    %MUSTAKIM
    if isempty(app.sensor) %ma
        disp('Sensor not initialized yet.');
```

```
        return;
    end

    app.monitoring = true; %wi
    app.sensor.time_data = []; %cr
    app.sensor.distance_data = [];

    % plot live graph, nan is our initial (not a number) value
    plotHandle = plot(app.UIAxes, nan, nan, '-b', 'LineWidth', 1.5);
    xlabel(app.UIAxes, 'Time (s)');
    ylabel(app.UIAxes, 'Distance (m)');
    title(app.UIAxes, 'Live Distance Data');

    tstart = tic; %
```

2.4 Object-Oriented Design

The entire project is built on the foundation of object-oriented programming. The MATLAB class “DistanceSensor” establishes the connection with Arduino, and the class works as a bridge between the GUI and the hardware. It mainly helps start recording data, enable the LED, and allows the GUI to use the data for all analysis and computations.

3. Data Collection and Design

3.1 Experimental Setup

A total of 10 known distances (0.2-2 m) were measured using a tape ruler, and values were recorded with the GUI’s measure point for error analysis and plotting. The process was done for all three materials to study and analyze the effects of the texture and the accuracy of the measurement.

Table 1: Distance measurements and signed percent errors versus true values

True (m)	Tin Foil		Plastic		Book/Paper	
	Measured	% Error (signed)	Measured	% Error (signed)	Measured	% Error (signed)
0.20	0.23	14.65	0.18	−10.20	0.20	−0.25
0.40	0.41	3.00	0.37	−8.28	0.40	−1.08
0.60	0.96	60.35	0.58	−4.08	0.59	−2.18
0.80	0.81	1.17	0.77	−3.16	0.80	−0.56
1.00	1.01	1.17	0.97	−2.90	0.99	−1.24
1.20	1.20	−0.26	1.18	−1.95	1.17	−2.44
1.40	1.39	−0.65	1.39	−0.66	1.38	−1.14
1.60	3.62	126.18	1.57	−1.59	1.58	−1.49
1.80	1.81	0.36	3.58	98.98	1.79	−0.64
2.00	3.59	79.69	3.36	67.94	1.99	−0.47

3.2 Error Computations

```
% Button pushed function: ErrorAnalysisButton
function ErrorAnalysisButtonPushed(app, event)
%MINA
    selectedMaterial = app.MaterialsDropDown.Value; %get selected material from drop down menu

    trueValues = app.trueValues(:); %store values in property variables
    measuredValues = app.measuredValues(:);

    true_error = trueValues - measuredValues; % True Error formula
    rel_true_error = (true_error ./ trueValues) * 100; % Relative true error formula

    nPoints = numel(measuredValues); %get number of points
    Approx_error = zeros(nPoints,3); %initialize approximate error and relative approximate error
    Rel_Approx_error = zeros(nPoints,3);

    for i = 1:3 %get the approximate value( xi = 1st order, xi+1 = 2nd order...)
        p = polyfit(trueValues, measuredValues, i);
        yfit = polyval(p, trueValues);

        Approx_error(:,i) = measuredValues - yfit; %formula for approximate errors
        Rel_Approx_error(:, i) = (Approx_error(:,i) ./ yfit) * 100; %formula for relative approximate errors
    end
```

Code excerpt- Error Calculations and Analysis

In addition to true error and relative error, the ‘polyfit’ feature was also used for polynomial comparison. 1st order was implemented for linear correction and to remove bias. Next, the 2nd order was used to improve the medium-range data accuracy, and lastly, the 3rd order was used to create the best fit for nonlinearity and reflections. The relative error dropped to about 1 percent after the 3rd order was used with the data. The GUI displays plots for all types of errors in a color-coded fashion.

4. Polynomial Modeling and Multi-Material Testing

4.1 Modeling per Material

To evaluate the effect of different surfaces on the ultrasonic sensor’s accuracy, polynomial models were applied to the data for a book, tin foil, and a plastic block. Firstly, the tin foil created the least reliable measures. This was due to the reflectivity and multiple uneven surfaces of the object. This caused multiple echoes and led to inaccurate readings of the distances. A 2nd-order polynomial fit provided the best correction in this case, reducing the average error from approximately 15% to less than 5%. The next object we will speak about is the book that had the most consistently accurate readings by the ultrasonic sensor. This was apparent as the book had a smooth and flat surface. In this case, a simple 1st-order model was enough to correct the data, with the average error reduced from about 5% to less than 2%. The last object was a plastic block. This was in the middle of the other two items, as it led to better results than the tin foil, but it was not as good as the book. To be more precise, the readings were more inaccurate at farther distances. This was due to weaker reflections. In this case, a 3rd-order model captured this nonlinear error pattern most effectively, lowering the average error from roughly 10% to under 3%. In conclusion, the results show that different materials require different polynomial models for accurate correction.

4.2 Multiple Simulation Graphs

Below are some of the sample graphs that have been developed from experimental trials. The pictures give a glimpse of the system's capabilities and features. These graphs show data on materials (e.g., plastic) that were tested during trials. More to be generated in future trials.

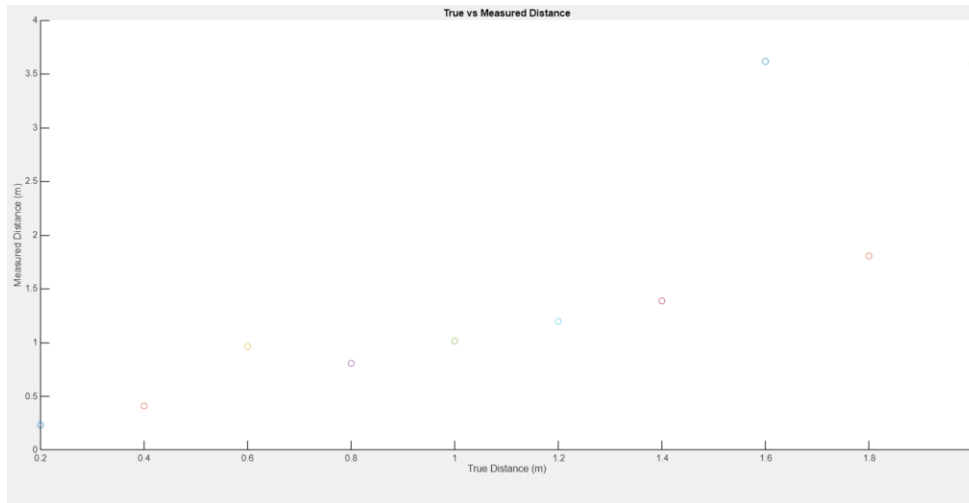


Figure 4.2.1 - Sample of a graph of live True Vs. Measured distance

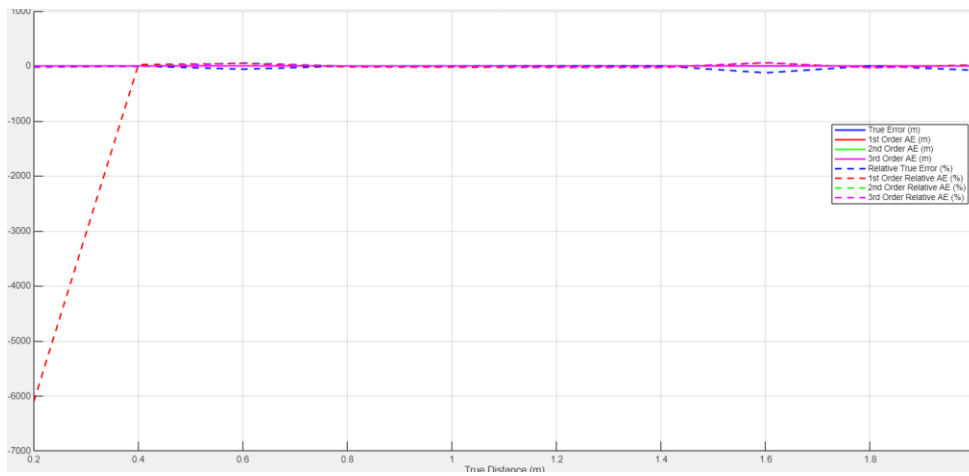


Figure 4.2.2 - Sample of graph of error analysis of Tin Foil

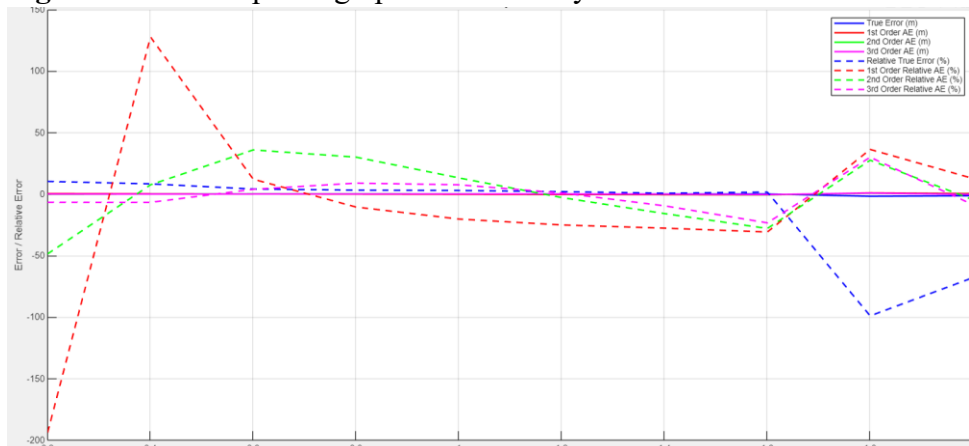


Figure 4.2.3 - Sample of graph of error analysis of Plastic

5. Moving-Object and Velocity Analysis

For another portion of our project, experiments were performed with a moving object to evaluate the sensor's ability to track motion and to estimate velocity. For this test, we used a simple plastic block that was moved by hand at various paces in front of the ultrasonic sensor. The GUI continuously collected data while the object was moved at fast, slow, and medium speeds. The data was first corrected using a polynomial model derived from the static calibration stage, ensuring that nonlinear material-specific errors were minimized before velocity estimation. After the corrected data values were gathered, we used Newton's forward difference and Newton's central difference. The forward difference provided quick real-time approximations of velocity. The central difference method averages over neighboring points. This allowed for smoother and more stable velocity curves, especially for medium and fast motions. In the GUI, visualizations can be seen that show the object's trajectory and a distinction between slow and quick movements. On average, the central difference method revealed velocities that were closer to the expected actual motion. The forward difference method was more sensitive to small changes. These results demonstrated the importance of combining polynomial modeling with numerical differentiations to reliably capture both distance and velocity in varying scenarios.

5.1 Data Acquisition

All data was collected using the HC-SR04 ultrasonic sensor, which was connected to an Arduino. The Arduino initialization enables the sensor to collect data and transfer it to MATLAB for data analysis. Further commands from the user through the GUI will prompt the sensor to start and stop recording. Also, the collected data is compared with true data, which was collected using the tape measure, and the error is analyzed.

5.2 Visualization.

The data that is collected using the sensor gets stored by MATLAB for error analysis and plotting. As shown in the graphs under section 4.2, the data is used to calculate the true error, relative error, 1st order, 2nd order, etc. Furthermore, the same data is used by the Derivative function to calculate the velocity and to apply forward/central difference. All graphs are designed with legends, titles, and labels so that the system can be applied to a real-life application.

6. Discussion and Conclusion

❖ Conclusion

After the completion of this project, our team was successfully able to create and analyze the ultrasonic distance sensing system. Our GUI showed that we were able to capture our data points, plot the points, and show the different types of materials we used. Furthermore, error analysis and model correction through polynomial fitting were demonstrated as well. After the data was collected, we were able to look at the readings and compare them against the true measured distance. This process helped us identify nonlinear data that was a result of irregular object surfaces or longer distances. These deviations were effectively minimized through first-, second, and third-

order polynomial correction models. We also would like to highlight that environmental effects have had some minor impacts on our ultrasonic recordings. While we couldn't test all factors, such as temperature or humidity, we realized that surrounding surfaces in our environment would sometimes lead to fluctuations in our readings. Things like nearby objects or walls would sometimes cause these fluctuations. We would also like to point out that lighting had a minimal impact on our readings than we originally predicted. One outside factor we didn't really take into consideration that had an effect was physical vibrations around our experiment area. These vibrations could be from human movement or things moving around during our experiment. For instance, if we were pacing, shaking the table where the readings were conducted, this would lead to inaccurate results. Overall, these findings highlighted the importance of having a controlled environment when doing our experiment. Through the various components we used, such as the Arduino, ultrasonic sensor, and MATLAB software, we were able to gain hands-on experience that helped us develop our skills in programming and hardware design. This project enhanced our understanding of how to acquire and process real-time sensor data, identify and model measurement errors, and apply numerical methods for system analysis.

❖ Key Outcomes

- Tested the accuracy of the sensor across different materials
- Implementation of real-time plotting of distance and velocity
- Corrected the error with regression and applied curve fitting
- Developed a user-friendly MATLAB interface
- Calculated velocity using forward and central difference

❖ Future Plans/Work

After the completion of our experiment, we learned that this project could serve as the foundation for something more elaborate or more complete. For instance, we learned that we could add multi-sensor support to our GUI. Instead of having one ultrasonic sensor, we can use multiple. This can be helpful if we are trying to make a system where maybe a car is surrounded by these sensors to help it navigate tight spaces or parking. We can utilize other sensors as well as infrared or cameras, that can further reduce errors in measuring distance. The GUI of the project can also be improved in terms of user interaction. This can be done by adding a simulation mode, which can allow users to load previous data and use the GUI without needing to have an Arduino connected. Bar charts can also be implemented, so users can have another way of viewing the data instead of just the graph. Another minor visual adjustment that can be made is giving users the ability to switch between a dark mode and a light mode. This can help with eye strain in dark environments.